

Emuqu — Build-Time Governance of a Consumer Physiology Score

Constants-hash version locks, a machine-checked science register, gates that must prove they fail, and one-way budget ratchets

Technical White Paper • v1.3 • 8 October 2026

Chris Sharp • github.com/chrissharp80

Describes Emuqu (formerly Flow HRV, then Flow Recovery) at source revision e028039 (2026-10-07), scoring version v3.1.oct2026. Emuqu is a consumer wellness app, not a medical device.

Cite as: Sharp C. Emuqu — Build-Time Governance of a Consumer Physiology Score. Technical White Paper v1.3, 8 October 2026. github.com/chrissharp80/emuqu, docs/whitepapers/04-claims-governance.md.

Executive Summary

Emuqu is an iOS app that turns overnight beat-to-beat (RR) intervals from a chest strap, together with sleep and vitals data, into a daily recovery score. Like most consumer composite scores, its weights and bands are hand-calibrated: some inputs rest on published methods, others are product rules whose evidence status the app records openly. Keeping such a score consistent with its evidence means preventing four kinds of quiet drift: a scoring constant changes and stored history mixes two algorithms under one name; a heuristic's evidence status is forgotten and copy starts to imply more than the evidence supports; a quality gate reports "clean" while measuring nothing; and a debt ceiling is raised to make the build pass.

Emuqu answers each with build-time checks, shell and Python scripts that almost all run in continuous integration (CI); the test-mutation harness runs by hand on an iOS simulator.

- **A constants-hash version lock.** The 155 numeric constants of two designated scoring files are hashed with SHA-256 (Secure Hash Algorithm, 256-bit). The build fails if the digest moves while the scoring version stays the same. Every stored score carries its version; older records decode to an explicit `unversioned` sentinel.
- **A machine-checked science register.** A JSON (JavaScript Object Notation) inventory classifies 19 heuristics into a closed status vocabulary defined in the register itself: 3 `validated` (the register's label for a use backed by a peer-reviewed result or checked against reference data on every build), 4 `supported-transfer` and 12 `awaiting-validation`. Each entry states the wording copy may and may not use, and every score-adjusting function found by a name scan must be claimed or waived.
- **Gates that test the gates.** Each gate that needs no build must go red on a planted violation (64 plants), fail closed on temp-file errors, and be named in the full-CI workflow. A separate harness applies 125 targeted mutations that named test suites must catch.

- **One-way budget ratchets.** Each CI budget file may move only in the tightening direction its filename declares, unless a commit trailer acknowledges the loosening.

The parts form one chain: a change to a constant in a designated scoring file forces a version change, which forces an edit of the register's permitted wording, which the regulated-claim perimeter of Paper 3, *A Measurement-Grounded Assistant with a Multilingual Regulated-Claim Perimeter* ([link](#)), polices for prohibited vocabulary. Section 4 compares the published tools this builds on, and Section 5 states what is new.

Section 5 gives the dated record for each statement and Section 8 the provenance; the earliest form of each was publicly disclosed on 2026-09-08. Section 6 lists the counts the gates themselves print at this revision.

1. The Problem

1.1 Composite scores are calibrated, not validated

Consumer wearables publish composite "readiness" or "recovery" numbers built from heart-rate variability (HRV), sleep and other signals. The component measurements have a literature: the Task Force standards for time-domain HRV [1], and the smallest-worthwhile-change method on a rolling baseline of $\ln(\text{RMSSD})$, the log of the root mean square of successive RR differences [2]. The way the components are combined usually does not. Reviews of consumer wearables in athletic populations note that the proprietary algorithms are rarely validated independently [3]. Some popular sub-heuristics have been challenged directly: the ratio of low-frequency to high-frequency power (LF/HF) as an index of "sympathovagal balance" [4], and the acute:chronic workload ratio (ACWR) as an injury-risk signal [5]. Emuqu's own register records that its composite weights are calibrated, not outcome-validated.

A product in this position can keep its claims honest in two ways. It can remove every rule without outcome evidence, which in practice removes the product. Or it can keep bounded heuristics and make their status impossible to forget. Emuqu takes the second route, which works only if forgetting is mechanically prevented. That mechanism is what this paper describes.

1.2 Sources of drift

Table 1.1. Ways a governed score drifts, and why code review alone misses each.

Drift	How it happens	Why review alone misses it
Algorithm mixing	A threshold or weight is retuned; stored scores computed before and after look identical	The diff touches one number, and nothing marks history
Claim creep	Copy is reworded ("strong predictor" becomes "strongly suggests")	A lexical banned-word list matches phrasings, not claims
Dead evidence	A test cited as backing for a rule is deleted or renamed	Nobody re-reads the evidence inventory
Resurrected errors	A figure corrected in one file survives in another	Corrections are local; copies are not
Fail-open gates	A temp file cannot be created; the gate greps an empty file and prints "clean"	A green tick looks the same whether or not anything was measured

Drift	How it happens	Why review alone misses it
Unwired gates	A gate exists in the scripts directory but no CI step runs it	The directory listing looks complete
Goalpost moves	A budget file is edited upward so CI passes	The edit is one character

Each row is a failure a code reviewer reads straight past, because the change that causes it is small and locally reasonable. The common answer is to make each one a build failure.

1.3 What existing approaches provide

Several established tool families each solve part of this problem: lint baselines and coverage ratchets, model cards and model registries, mutation testing, requirements-traceability tools, checks that test other checks, and the traceability required by medical-device quality systems. Section 4 compares each with Emuqu, and Section 5 states what Emuqu adds to them.

1.4 The approach

Emuqu binds three things together with executable checks: the numbers in the scoring code, the version string on stored scores, and the evidence status and permitted wording of each heuristic. This build-time governance complements Paper 5, *Blind-Audit Convergence: A Method for Directing AI to Produce Converged Software Designs* ([link](#)). Paper 5 governs how the design and its claims are reached; this paper keeps them true once the code exists.

2. Architecture Overview

The governance layer is a set of scripts in the repository, run by two CI workflows and by a local `make ci` target. The gates never run on the user's device; the on-device parts are the version stamp on stored scores and the text that shows it.

Table 2.1. Governance operations by phase.

Phase	Operation	Purpose
Author time	Change a scoring constant	—
Gate: scoring governance	Hash numeric constants; compare with the recorded version and hash line	Force a version bump when the numbers move
Gate: scoring governance	Scan app sources for the ranking-weight identifier outside one module	Keep a hand-set ordering key from being displayed as a confidence
Runtime	Stamp every new score breakdown with the current version; decode unstamped history as <code>unversioned</code>	Keep each stored score attributable to its algorithm
Gate: science register	Register version must equal code version	Force an edit of the register on every version bump

Phase	Operation	Purpose
Gate: science register	Status vocabulary, required fields, test paths, retracted claims, discovery scan, entry floor	Keep the claim inventory complete in a checkable sense and keep it current
Gate: regulated-claim perimeter (Paper 3)	Prohibited-term lint on Swift string literals and string catalogs; build-time list reconciled with the runtime assistant guards	Keep medical-claim vocabulary out of user-facing text
Meta-gates	Planted violation per gate; fail-closed temp files; wiring check	Prove each gate goes red, fails closed and is named in the full-CI workflow
Test verification	Targeted mutation per behavior; leak backstop; tree lock	Prove the test suites assert the behaviors that matter
Ratchets	Direction-aware budget comparison against a baseline commit	Debt only falls, floors only rise
Reporting	Evidence report prints live counts	Give reviewers numbers that cannot be stale

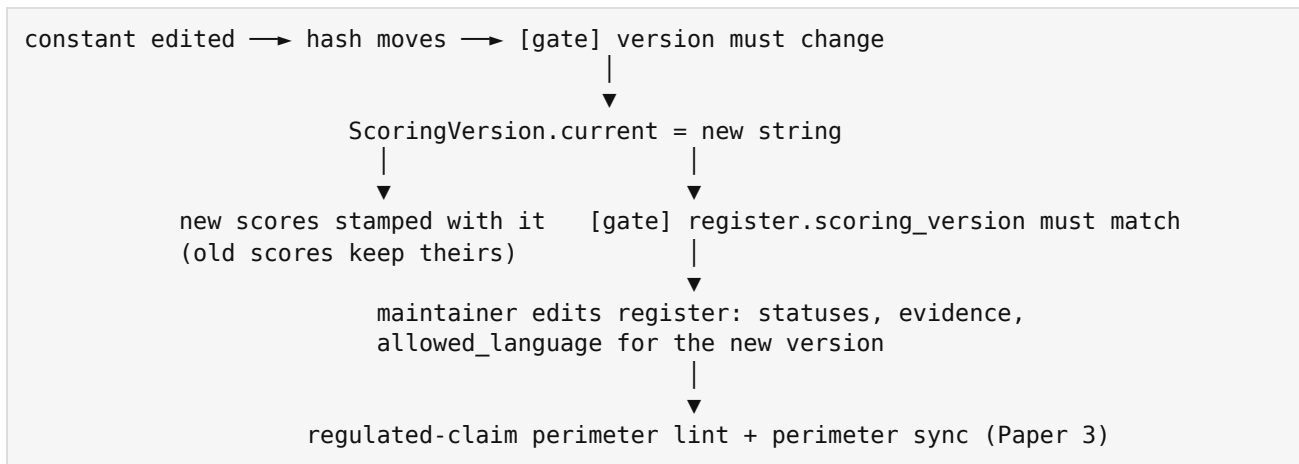


Figure 2.1. The chain from a scoring-constant edit to the wording users see.

3. Methods

3.1 Scoring-constants hash and version lock

3.1.1 Hash and version lock

Inputs.

- Two designated scoring-constant source files: the recovery-score constants file (91 numeric declarations) and the HRV thresholds file (64 numeric declarations), 155 in all at this revision.
- The scoring-version source file, which declares `current` and `unversioned`.
- A one-line baseline file in the CI directory containing `<recorded_version> <recorded_hash>`. At this revision it reads `v3.1.oct2026 541b56e600080795`.

Steps.

1. Visit the two constant files in a fixed order: recovery-score constants first, HRV thresholds second. If either file is missing, exit 2 (configuration error).
2. Read each file and delete every `//` line comment (the regex `//[^\n]*`, replaced with nothing). Rewording a rationale comment therefore never changes the hash. Block comments are not stripped; neither file contains one.
3. Extract all matches of `static let\s+(\w+)\s*:\s*\s*(\w<>[\], .)*=\s*([-\.d.]+)` giving `(symbol, value)` pairs. `value` is the literal text of the number as written, so `60` and `60.0` are different values. A declaration whose right-hand side is not a bare numeric literal is not matched.
4. Sort the pairs of each file lexicographically by `(symbol, value)`. Moving a declaration within a file, or renaming an enclosing namespace, does not change the hash. Renaming a symbol or changing its value does. Symbols repeated under different nested namespaces are all included.
5. For each pair, in file order and then sorted order, feed the UTF-8 bytes of `"<file path>:<symbol>=<value>\n"` into one running SHA-256 digest.
6. Take the first 16 hexadecimal characters of the digest (64 bits) as `current_hash`.
7. Read `current` from the version file with `static let current\s*=\s*"([^\"]+)"`. If it cannot be read, exit 2.
8. If the baseline file is missing, exit 2. Otherwise split it on whitespace into `recorded_version` and `recorded_hash`.
9. Print the number of constants hashed, the hash and the version, whatever the result.
10. **Violation rule:** if `current_hash != recorded_hash` **and** `version == recorded_version`, report "scoring constants changed while the version stayed the same" and exit 1.

Why it works. A score's meaning is the set of numbers that produced it, so the version should change exactly when those numbers change. The hash captures the numbers and nothing else. Stripping comments means the rationale written beside a constant can be improved freely without a version bump. Sorting per file means reorganizing code is free too, while a renamed or retuned constant is not. Hashing the literal text rather than a parsed value is the conservative choice: `60` becoming `60.0` is treated as a change, so the gate never has to decide whether two spellings are equal. The gate does not bump the version itself. A version is a name stamped on users' data, and requiring a person to choose the new name makes each algorithm change a deliberate, reviewable act. Sixteen hexadecimal characters keep the baseline on one human-readable line while leaving accidental collisions out of practical reach. Exit code 2 for a missing input separates "cannot measure" from "measured a violation", so neither is ever reported as clean.

3.1.2 Ranking-weight confinement

The same script also confines a ranking key. The probable-cause list orders candidate causes by a hand-set number named `rankingWeight`. Its register entry records that the value has no outcome evidence and must never be rendered as a confidence or probability. The gate walks every Swift file under the app sources. Any line containing the substring `rankingWeight` in a file outside the cause-detection directory is a violation.

Why it works. An ordering key is a legitimate internal tool, but the same number shown as "72%" reads as a probability. Confining the identifier to one directory keeps it out of views, exports, analytics and accessibility text

at the point of use. Moving it elsewhere requires a rename, and a rename is a visible change that a reviewer reads against the register entry.

3.1.3 Version stamp at runtime

Table 3.1. How the scoring version travels with each stored score.

Rule	Behavior
Construction	A new <code>ScoreBreakdown</code> defaults its <code>scoringVersion</code> field to the current version
Decoding	A stored breakdown with no <code>scoringVersion</code> key decodes to the literal string <code>unversioned</code> , never to the current version. Scores from before the stamp may come from the v1 algorithm (which preceded v2.may2026) or from v2.may2026, because the app offered a v1-to-v2 history recompute the user may or may not have run.
Derived copies	Code that rebuilds a breakdown copies the version field forward. This covers the vitals refresh after scoring and the stripped copy prepared for iCloud sync.
Display	The recovery-score detail screen shows "Scored by v3.1.oct2026" (the stored value), or "Scored before score versions were recorded" for <code>unversioned</code> . Beneath it is a fixed line saying the inputs are research-informed and the weights and bands are calibrated, not outcome-validated.
Assistant	The in-app assistant's fact catalog (key <code>score.algorithm.version</code>) and its context block report the current version string and describe the v1 weighting older history may carry.

Why it works. Freezing and versioning each stored score keeps history attributable: a user comparing this month with last month can see whether the same algorithm produced both. The `unversioned` sentinel is the honest default for old records. Their true version is unknowable, because a recompute may or may not have run, and labeling them with the current version would assert something false. Carrying the stamp through every derived copy matters because a stamp dropped on sync or refresh would quietly relabel old scores as new. Placing the stamp next to the fixed "calibrated, not outcome-validated" line puts the version and the evidence status in front of the user together.

Table 3.2. Scoring versions in the public repository.

Version	Introduced
v2.may2026	2026-09-08 (public initial commit)
v3.oct2026	2026-10-03
v3.1.oct2026	2026-10-04

3.2 The science register

3.2.1 Data model

One JSON file holds the register. Its top-level keys:

Table 3.3. Top-level keys of the register.

Key	Content
<code>_purpose</code>	Prose: a governed inventory of heuristics that change a number the user sees
<code>_what_the_gate_can_and_cannot_prove</code>	Prose statement of the gate's scope (quoted in Section 7)
<code>_retracted_claims</code>	Object whose <code>claims</code> list holds evidence phrasings that were once stated and later corrected (3 strings at this revision: two spellings of one reliability range, and one citation to a replication study)
<code>_status_meanings</code>	Map from each allowed status word to its definition. The gate reads the vocabulary from this map.
<code>_waived_symbols</code>	Map from function name to the reason it is not an entry (1 at this revision: a display-only translation helper)
<code>scoring_version</code>	Must equal the code's current version
<code>entries</code>	List of classified heuristics

Table 3.4. Status vocabulary (exact words; meanings abridged from the register).

Status	Meaning	Count
<code>validated</code>	An external peer-reviewed result supports this specific use, or the repository verifies it against reference data on every build	3
<code>supported-transfer</code>	A real published result exists, but the app applies it outside the population, protocol or endpoint it was established in. Defensible, not established.	4
<code>awaiting-validation</code>	A hand-calibrated product rule with no outcome evidence for these particular numbers. Retained because it is bounded and removing it would change stored scores; version-stamped so a later removal is visible.	12
<code>scheduled-for-removal</code>	Kept only until a replacement lands	0

Table 3.5. Register entries at this revision.

Status	Entries
<code>validated</code>	<code>hrv-time-domain-arithmetic</code> , <code>lnrmssd-swc-band</code> , <code>dfa-artifact-rejection</code>
<code>supported-transfer</code>	<code>dfa-a1-exercise-threshold</code> , <code>hrr-12bpm-band</code> , <code>training-load-ladder</code> , <code>tanaka-hrmax</code>
<code>awaiting-validation</code>	<code>resting-dfa-a1-reference-band</code> , <code>lfhf-window-filter</code> , <code>pns-sns-gap-readiness</code> , <code>acwr-readiness-damper</code> , <code>composite-weights-60-25-15</code> , <code>sleep-score-six-factor</code> , <code>hrv-sleep-staging</code> , <code>vitals-illness-pattern</code> , <code>spo2-flat-penalty</code> , <code>cause-ranking-weights</code> , <code>rmssd-absolute-fallback-bands</code> , <code>baseline-staleness-penalty</code>
<code>scheduled-for-removal</code>	none

In these identifiers, `dfa-a1` is the short-term scaling exponent α_1 of detrended fluctuation analysis (DFA); `hrr` is heart-rate recovery; `lfhf` is the LF/HF ratio; `pns-sns` is the parasympathetic-minus-sympathetic index gap; `acwr` is the ACWR; and `spo2` is blood-oxygen saturation (SpO_2).

Table 3.6. Entry fields.

Field	Required by the gate	Role
<code>id</code>	Unique	Stable name
<code>status</code>	Must be a key of <code>_status_meanings</code>	Evidence level
<code>affects</code>	Non-empty	Which user-visible outputs change
<code>evidence</code>	Non-empty	What supports or does not support the rule, with sources
<code>allowed_language</code>	Non-empty	The wording copy may use, and the wording it must never use (for example "Damper, bookkeeping. Never injury risk or dangerous.")
<code>tests</code>	Optional; every path listed must exist	Tests that pin the behavior
<code>symbols</code>	Optional	Function names in the analysis layer that this entry claims
<code>notes</code>	Optional	Free text; scanned for retracted claims

3.2.2 Register gate

- Exit 2 if the register, the entry-floor file or the version source is missing, if the JSON does not parse, or if `_status_meanings` is empty.
- For each entry, record a problem for a duplicate `id`, a status outside the vocabulary, an empty `affects`, `evidence` or `allowed_language`, or a `tests` path that does not exist in the working tree.
- Read `current` from the version source and record a problem if the register's `scoring_version` differs. Every version bump therefore fails this gate until the register is edited.
- Retracted claims.** For each retracted string, search for an exact substring match in:
 - each entry's `evidence`, `affects`, `allowed_language` and `notes` fields (lists joined with spaces);
 - every `.swift` and `.md` file under the app sources and the docs directory.

The retracted list itself is the only place the strings may appear, which is why this paper describes them without quoting them.

- Discovery.** Over every Swift file in the analysis directory, excluding the cause-detection subdirectory, collect the names matched by `func\s+([a-zA-Z]+(?:Adjustment|Penalty|Bonus|Modifier|Damper|Fraction))\b`. Discard names ending in `Descriptions` or `Message`. At this revision the scan finds 13 functions: 12 claimed and 1 waived.
- Let `claimed` be the union of all entries' `symbols`, and `waived` the non-underscore keys of `_waived_symbols`.
 - Each discovered name in neither set is a problem ("a score-adjusting function no register entry claims").

- Each name in `claimed` or `waived` that was not discovered is a problem ("claimed or waived but no longer exists"). A stale claim fails, just as a missing one does.

7. Print a tally by status, plus the discovered, claimed and waived counts. If there are any problems, exit 1.

8. **Entry floor.** Read the minimum entry count from the CI directory's register floor file. If the number of entries is below it, exit 1 with instructions: a heuristic still in the code must stay classified, and a heuristic removed from the code needs the floor lowered through the audited trailer of Section 3.6. If the count is above the floor, print a reminder to raise it.

The floor file's name begins with `min_`, so the ratchet of Section 3.6 treats it as a floor that may only rise.

3.2.3 Why it works

- *A closed vocabulary read from the register's own definitions.* A status word is only useful if everyone reads it the same way. Keeping each word's definition beside the word, and deriving the allowed set from that map, means no status can be used without a written meaning.
- *A per-entry wording boundary.* What may honestly be said about a heuristic depends on its evidence, and differs from one heuristic to the next. A global banned-word list cannot express "may say 'damper', may never say 'injury risk'" for one rule and something else for another. `allowed_language` puts that boundary in the same record as the evidence that justifies it.
- *Version agreement.* The moment the algorithm changes is the moment its evidence statements may stop being true. Requiring the register's version to match the code's turns every algorithm change into a forced visit to the register.
- *A retracted-claims list.* A correction made in one file does nothing about copies of the old figure elsewhere. Keeping the corrected phrasings in one list, and failing the build wherever else they appear, gives each correction a single home and makes it permanent.
- *Name-shape discovery with a symmetric stale check.* The analysis layer names its score-adjusting functions by what they do (`...Penalty`, `...Damper`). A scan over that convention catches a new adjustment on the day it is written, at negligible cost. The reverse check, that every claimed name still exists, keeps the register from describing code that is gone.
- *An entry floor.* Deleting an entry would silently shrink what the register makes visible. A floor that only rises makes every removal an audited, deliberate act.

3.3 The regulated-claim perimeter, briefly

Paper 3 covers the regulated-claim perimeter in detail. In outline, a Python linter reads a JSON list of prohibited regular expressions and scans every Swift string literal and string-catalog value for user-facing medical-claim vocabulary. Comments are stripped, and an allowlist covers the methodology page and the guard sources. A second gate proves that the runtime vocabulary used by the in-app assistant's input and output guards covers every build-time pattern: for each build-time regex it synthesizes a matching string and requires some runtime lexicon concept to match it too.

For this paper, the point is the division of labor. The perimeter is lexical: it bans phrasings across the whole app. The register's `allowed_language` field states the claim boundary per heuristic, which is a judgment about

meaning. The build forces the maintainer to revisit that boundary on every version bump (Section 3.2.2, step 3), and the perimeter enforces the vocabulary that no boundary permits. Each mechanism does the part it can do reliably.

3.4 Gates that test the gates

The repository has 45 gate scripts (names beginning `check_` or `enforce_`). Two meta-gates and one harness establish three properties of every gate: it **goes red** on a real violation, it **fails closed** when it cannot measure, and it **is wired**, meaning it is named in the full-CI workflow.

Why three properties. A green result means "nothing is wrong" only if the gate could have gone red, actually measured something, and actually ran. Each property fails independently: a gate with a typo in its pattern stays green on real violations; a gate whose temp file was not created greps nothing; a gate missing from the workflow never runs. Checking each property separately, for every gate, is what lets a green CI run be read as evidence.

3.4.1 Goes red: planted violations

The planted-violation harness holds a list of plants. Each plant is defined as `(name, gate command, file to mutate, mutation program)`. For each plant:

1. If a name filter was given and does not match, skip.
2. Run the gate. If it is already red, print SKIP ("fix that first") and continue. A red-after-mutation result proves nothing if the gate was red before.
3. Copy the target file to a backup directory, which was created through the fail-closed helper of Section 3.4.2.
4. Apply the mutation, a short Python program that edits the file. Most mutations assert that the original text occurs the expected number of times. If the mutation fails, print SKIP and restore.
5. Run the gate again, record its exit code, and restore every touched file.
6. A non-zero exit is PASS. A zero exit is FAIL ("STAYED GREEN — this gate protects nothing").

At this revision there are 64 plants. Examples: a retracted phrase appended to a register evidence field; an unknown register status; the hash baseline zeroed under the same version; a budget raised by 100000 with no trailer; a CI step renamed to a nonexistent script; the pNN50 mutation of Section 3.5.1 left in place; a bare `mktemp` in a gate. Several gates have more than one plant, one for each shape of miss the gate is meant to catch.

Every gate has a plant. After the plants run, the harness lists every `check_*.sh` and `enforce_*.sh` in the scripts directory and drops two groups:

- a two-entry `NEEDS_BUILD` exemption list: the coverage-floor gate and the Thread Sanitizer gate, which need a build or a simulator;
- the preflight gate of Section 3.4.2, which the harness classifies as the preflight library's self-test rather than as a gate. It still has two plants of its own.

For each remaining gate, the harness greps its own source for the gate's filename. If any filename is missing, the run exits 1 and lists the unplanted gates, so adding a gate and adding its plant must happen in the same change.

The run also exits 1 if any plant was FAIL. A plant that was SKIPped is reported and does not fail this harness; the test-mutation harness of Section 3.5.1 treats skips more strictly.

Why it works. The only convincing evidence that a check can detect a violation is a violation it detected. Requiring green first rules out a gate that is red for an unrelated reason. The completeness check is what makes the discipline scale: the set of gates that must have plants is derived from the directory, not from anyone's memory, so coverage grows with every new gate automatically.

3.4.2 Fails closed: temp-file preflight

The gates run with `set -uo pipefail` but not `-e`, because several rely on `grep` returning non-zero when nothing matches. Without `-e`, a failed `mktemp` leaves an empty path, and later reads of that path find nothing, which a gate would report as "clean". The design has two halves.

Helper library.

- `gate_mktemp label` creates `${TMPDIR:-/tmp}/label.XXXXXX` with an explicit template, which behaves the same under BSD and GNU `mktemp`. If creation fails, the path is empty, or the file is not writable, it prints the reason and calls `exit 70 (EX_OSERR)`.
- `gate_mktemp_dir` does the same for directories.
- `gate_require_nonempty` exits 70 when an input the gate is about to parse is missing or empty.

The temp-file helpers are called inside `$( )`, which is a subshell, so their `exit` cannot stop the caller. Every such call site must therefore append `|| exit $?`.

Preflight gate. For every `.sh` file in the scripts directory except itself, it scans each non-comment line and reports:

1. any bare `mktemp` (one not part of a `gate_mktemp` call);
2. any line calling `gate_mktemp` without `|| exit` on the same line;
3. any `source "${dirname "$0"}/<path>"` whose target does not exist, because a mistyped source path would silently define no helpers.

Any finding exits 1. At this revision it scans 53 scripts and finds 10 guarded temp-file uses.

The scoring and register gates also fail closed on configuration problems: a missing input exits 2, not 0. In CI, the ratchet treats a missing baseline as a failure (Section 3.6.2).

Why it works. Fail-open behavior comes from one predictable place, an unchecked resource whose absence looks like an empty result. Routing every temp-file creation through one helper puts the failure handling in a single audited function, and the static scan makes that routing mandatory. Checking the `|| exit` guard at the call site closes the subshell gap that would otherwise let the helper's exit vanish.

3.4.3 Is wired: CI reachability

The wiring gate reads the full-CI workflow file (`.github/workflows/ci.yml`), which runs on demand and is called by the release workflow. The path can be overridden by an environment variable; a missing file exits 2. Every `check_*.sh` or `enforce_*.sh` whose filename does not occur as a substring of that file is reported. One

gate is exempt by name with a recorded reason: the coverage-floor gate, which runs inside the coverage step. Any unwired gate exits 1. At this revision 44 gates are named in the workflow and 1 is invoked by the coverage step.

Why it works. A gate that exists but never runs protects nothing and looks complete. Deriving the list of gates from the directory and the list of steps from the workflow makes reachability a property the build checks, rather than something a reviewer infers.

3.5 Tests that must catch defects

3.5.1 Targeted test mutations

This harness applies the planted-violation discipline to the unit tests instead of the gates. It holds a curated list of 125 mutations, each defined as (name, test suite identifier, production file, mutation program). Examples: raise the pNN50 threshold (the share of successive RR differences above a limit) from 50 ms to 75 ms; return the mean instead of the standard deviation for SDNN (the standard deviation of normal-to-normal intervals); feed raw RR intervals instead of successive differences to RMSSD; shift the top verdict band from 90–100 to 91–100; remove the finiteness guard on a date-to-milliseconds conversion; drop the detrend step from DFA.

For each mutation:

1. Run the named suite alone (`-only-testing`, parallel testing off) and classify the result:
 - **0**: passed;
 - **1**: tests failed;
 - **2**: build or runner failure, recognized by a build-failure banner in the log or by the absence of any "Test Suite" or "Test Case" line.
2. If the baseline result is 2, retry once. If the baseline is still not 0, print SKIP with the reason.
3. Back up the file and apply the mutation. Each mutation asserts that its original text occurs exactly once, so a mutation cannot silently land in a doc comment or the wrong place. If it cannot be applied, print SKIP.
4. Run the suite and restore.
5. A non-zero result is PASS ("caught"). A zero result is FAIL ("SURVIVED — asserts nothing about this").
6. At the end, **any FAIL or any SKIP exits 1**. "A skip is not a pass."

This harness needs a simulator and a build per mutation. It is not part of `make ci`, and no CI workflow runs it; it is run by hand when analysis or storage code changes.

Why it works. For a physiology score, the important question is not what fraction of random mutants die but whether a specific wrong formula would be caught. Each curated mutation is a plausible scientific error (the wrong threshold, the wrong statistic, the missing detrend), and naming its suite turns a survivor into a precise finding: this suite does not assert this behavior. Running the suite alone keeps attribution exact. Classifying build failures separately stops a broken build from counting as a catch, and treating every skip as a failure stops an unapplied mutation from passing silently.

3.5.2 Leaked-mutation detection and the tree lock

Both harnesses edit tracked files in place and restore them in an EXIT trap. A killed process skips the trap, so two backstops cover that case.

Leak detector. A static gate parses the test-mutation harness's source:

1. Split the source on the literal line prefix `mutate "`.
2. For each case, take the target path: the first double-quoted string that starts with the app directory and ends in `.swift`. Take the original text from either `o='...';assert` or `s.count('...')` followed by a comparison operator.
3. Unescape `\'`, `\"`, `\n` and `\$`. Resolve `' + chr(N) + '` concatenations to the character `N`.
4. If the original still contains `chr(` or a string concatenation, mark the case unparsed.
5. If the target file is missing, or the original text is not present in it, report a leak.
6. Any leak exits 1. **Any unparsed case also exits 1**, because a case the parser cannot read is a case it is not checking.

This gate runs on every push to main and every pull request.

Tree lock. Before mutating, each harness atomically creates a lock directory inside the git directory using `mkdir`, and writes its name and process ID (PID) into it. A second mutator finds the lock; if the PID is alive, it exits 75 (`EX_TEMPFAIL`). If the PID is dead, it warns that the earlier run never restored, points to the leak detector, and takes the lock over. The helper library also provides a reader function that refuses to run (exit 75) while a live mutator holds the lock; the local CI simulation (`scripts/simulate_ci.sh`, run by `make ci-local`) calls it. The lock is released in the same trap that restores files.

Why it works. The mutation list already states, for every case, the exact text that must be present in the clean tree. Reading that list statically turns it into a manifest that exists before any run, so no run has to record one, and a leak from a run killed at any point is still found. Because the detector needs only Python and the repository, it runs on Linux for every push to main and every pull request, far more often than the harness itself. Creating a directory with `mkdir` is atomic, and keeping it inside the git directory keeps it out of the working tree and out of commits.

3.6 One-way budget ratchets

3.6.1 Policy and counting

Decision record. The repository's architecture decision record (ADR) on ratcheted budgets, ADR 004, states the policy. Every quality rule has a measured numeric ceiling, never an aspirational one. A ceiling may only move down. There is no per-site escape hatch: the app sources contain no SwiftLint disable directives. The record reasons that a gate nobody can satisfy gets routed around, while a measured ceiling blocks regression immediately.

Counting code, not prose. Several budgets count occurrences of a pattern: static singleton reads, `try?`, legacy observable-object conformances. Before counting, the counter removes comments. Block comments `/* ... */` are blanked character for character, keeping newlines. Line comments `//...` are replaced with spaces of the same length, so nothing on an adjacent line is joined into a match. Explaining a piece of debt in a comment therefore never raises its count. A fixed list of platform singletons (`URLSession`, `FileManager`, `NotificationCenter` and others) is blanked before the singleton count, because no version of the app can inject those away.

3.6.2 Monotonicity gate

1. **Baseline resolution**, in order:

- an explicit argument;
 - the pull-request target branch (`origin/<GITHUB_BASE_REF>`);
 - a `BUDGET_BASE_REF` environment variable;
 - otherwise there is no baseline.
2. **No baseline.** If `BUDGET_REQUIRE_BASELINE` is set, exit 1 ("refusing to report enforcement that did not happen"). Otherwise print a skip notice and exit 0. The same applies to an all-zero SHA (a new or force-pushed branch) and to a ref that does not resolve.
 3. **Acknowledgments.** Collect them from every commit message in `baseline..HEAD`, using the case-insensitive regex `^budget-raise-ok:[[:space:]]*^[[:space:]]+`. Keep only the path token.
 4. **Comparison.** For each `*.txt` file directly in the CI directory, in sorted order:
 - Read the current value. It must match `^[0-9]+$` once whitespace is removed; otherwise skip the file.
 - Read the value at the baseline with `git show`. A file absent at the baseline is a new budget and is skipped. A non-integer baseline value is skipped too.
 - Equal values pass.
 - **Direction** comes from the filename. A basename starting with `min_` is a floor, and a decrease loosens it. Any other file is a ceiling, and an increase loosens it.
 - A loosening whose exact path appears among the acknowledged paths prints `ALLOWED`. Any other loosening is a violation. A tightening prints `ratcheted`.
 5. Any violation exits 1, with instructions to pay the debt or add `budget-raise-ok: <path> <reason>`.

CI wiring of the baseline. The full-CI workflow uses an explicit baseline input if one is given, and otherwise the parent commit. On a root commit there is no parent: if the budget files in the working tree have uncommitted edits, the workflow compares them with the root commit itself; otherwise it prints a notice and skips, because there is nothing to compare. Whenever a baseline exists, it sets the require flag, and a baseline that does not resolve to a commit is a hard failure. The per-push workflow passes the pull-request target on a pull request, and the parent commit with `BUDGET_REQUIRE_BASELINE=1` on a push to main. A push to main therefore cannot skip the comparison.

3.6.3 Documentation may not restate a ratcheting value

The documentation-links gate scans every Markdown file. Whenever a line names a CI budget file whose content is a plain integer, it looks for a bare 2–6-digit number in that line or the next one, using the regex `(?<![\w. ])(\d{2,6})(?![\w.%])`. Any such number is reported. The one exception is when the window contains a backtick code span with `cat`, `head` or `read_budget`, which is a live read and cannot go stale. Documentation must point at the file instead of quoting it.

3.6.4 Why it works

- *Measured, not aspirational, limits.* A ceiling set at today's measured value is satisfiable immediately, so nobody is tempted to route around it, and it blocks the next regression on the day it would land.
- *Direction in the filename.* The rule that governs a budget travels with the budget. A new budget declares its direction when it is created, there is no separate configuration that could drift from the files, and a reader can tell from a directory listing which numbers may rise and which may fall.

- *An audited escape, not a silent one.* Some loosening is legitimate, for example when a heuristic is deliberately removed. The trailer allows them, but only in the commit history, attached to the named file, where a reviewer and every later reader can see it.
- *A required baseline in CI.* A comparison that did not happen must not look like one that passed. Locally, the gate skips without a baseline so that uncommitted work needs no justification addressed to no one; in CI the `require` flag makes the same condition a failure.
- *No restated values in documentation.* A number copied into prose is a second copy that the ratchet cannot update. Pointing at the file keeps one source of truth.

Observed history. Across the 28 commits of the public repository (2026-09-08 to 2026-10-07), every integer budget that changed moved in its tightening direction. The SwiftLint warning, `try?`, test-skip and localization-orphan ceilings each fell more than once, and the refactor-spec watch ceiling fell to zero. The views-coverage floor rose. No commit message carries a `budget-raise-ok`: trailer.

3.7 Costed CI posture and the evidence report

GitHub Actions bills macOS minutes at ten times the Linux rate, so the plan's 3,000 billable minutes a month buy about 300 macOS minutes. The repository measured a full macOS run at about 108 wall-clock minutes, roughly 1,080 billable, so fewer than three full runs fit in a month. The posture follows from that arithmetic:

Table 3.7. CI workflows and what each runs.

Workflow	Trigger	Runner	Content
Script gates	Every push to main and every pull request	Linux	Every gate that needs only bash, Python and git: regulated-claim perimeter, perimeter sync, science register, scoring governance, localization, source-hygiene guards (including the leaked-mutation backstop), documentation and software bill of materials (SBOM) checks, wiring and preflight meta-gates, tech-debt budgets, and the monotonicity ratchet
Full CI	Manual, or called by the release workflow	macOS	Scope <code>gates</code> (about 14 billable minutes): SwiftLint and its budget, all budgets, the planted-violation harness and every gate step. Scope <code>unit</code> adds unit tests and the Thread Sanitizer. Scope <code>full</code> adds UI tests and coverage floors.
Local	<code>make ci</code>	Developer machine	Every gate, the planted-violation harness and the full test suite with coverage
Manual	By hand	Developer machine with simulator	Targeted test-mutation harness (Section 3.5.1)

The planted-violation harness runs in the full-CI workflow and in `make ci` but not in the Linux workflow, because several of its plants exercise SwiftLint, whose rule set differs on Linux and would make the warning budget unreliable.

The **evidence report** script prints, in seconds and without building, live counts (files, tests, locales, waivers) and every budget value, as text or JSON. It omits coverage and pass/fail results on purpose: those need a simulator, and a cached number would be stale.

Why it works. The hash lock, the register, the perimeter, the preflight and wiring meta-gates, the ratchet and the leak backstop need only bash, Python and git, so they run on Linux for every push to main and every pull request at almost no cost. The guarantees that matter most are therefore checked most often, and the expensive simulator work is spent where only a simulator can help.

3.8 How the parts form one chain

Figure 2.1 shows the sequence. A developer who edits a numeric threshold in a designated scoring-constant file fails scoring governance until the version changes; the new version fails the science register until its `scoring_version` and the affected entries' status, evidence and `allowed_language` are revisited; new or removed score-adjusting functions fail discovery or the stale-claim check; and new user-facing copy passes through the regulated-claim perimeter. Every gate involved is proven to go red, to fail closed and to be named in the full-CI workflow.

Why it works. Each gate is small and cheap. Their value is the coupling: the event that changes what a score means, a change to a hashed scoring constant, triggers in order a new name for the algorithm, a stamp on the user's data, and a forced review of what may be said about it.

4. Comparison with Existing Approaches

Emuqu builds on several established families of tools, each of which solves part of the problem. Lint baselines (SwiftLint's `--baseline` [6], betterer [7]) freeze existing violations so that new ones fail, and coverage ratchets (jest-ratchet [8], CoverageRatchet [9]) let a coverage threshold only rise. Model cards and model registries [10][11] attach documentation and version identifiers to machine-learning models. Mutation testing frameworks (PIT [12], Stryker [13]) mutate code broadly and report a mutation score. Requirements-traceability tools (sphinx-needs [14], Doorstop [15], reqcov [16]) keep requirements in the repository, restrict statuses to a configured list and link items to tests and code. Checks on checks (canfail [17], didrun [18] and restore-verified [19], all published on PyPI on 2026-08-31) require a check to go red on a deliberate break, to show evidence that it ran, and to restore mutated files verifiably. Medical-device quality systems (IEC 62304 [20], ISO 13485 [21]) require traceability and controlled change through audited process.

Table 4.1. Emuqu compared with existing approaches.

Approach	What it enforces	Direction of limits	Ties numbers to a version	Evidence or claim inventory	Proves its own checks fail	Source
SwiftLint baseline	New lint violations fail; recorded ones are tolerated	Not numeric; the baseline can be regenerated	No	No	No	[6]

Approach	What it enforces	Direction of limits	Ties numbers to a version	Evidence or claim inventory	Proves its own checks fail	Source
betterer	A test's result may not get worse than the stored result; improvements are written back	Per-test "better" comparator	No	No	No	[7]
jest-ratchet (npm, 2018)	Coverage thresholds rewritten upward as coverage improves	Coverage only rises; fixed by the tool	No	No	No	[8]
CoverageRatchet (NuGet)	Per-file coverage thresholds ratcheted automatically from Cobertura XML	Coverage only rises; fixed by the tool	No	No	No	[9]
Hash-without-version-bump CI checks (2026-09-17, 2026-09-27)	CI fails when hashed content changes and the semantic version does not	Not applicable	Yes, for the hashed files	No	No	[22][23]
Model cards	Structured documentation of intended use, evaluation and caveats	Not applicable	Documents a model version; not build-enforced	Yes, as prose	No	[10]
ML model registries (e.g. MLflow)	Versioned model artifacts with stage transitions	Not applicable	Versions artifacts; does not watch source constants	Metadata, free-form	No	[11]
sphinx-needs	Requirement items in documentation; status restricted to a configured list of names with descriptions; required fields by schema	Not applicable	No	Yes: typed items, statuses, links	No	[14]
Doorstop	Requirements as version-controlled text files linked to each other and to code and tests	Not applicable	No	Yes: requirement items and links	No	[15]

Approach	What it enforces	Direction of limits	Ties numbers to a version	Evidence or claim inventory	Proves its own checks fail	Source
reqcov (PyPI, 2026-09-04)	Requirements traced to tests and code; CI fails on uncovered requirements; traceability matrix	Not applicable	No	Yes: requirements with coverage status	No	[16]
PIT / Stryker mutation testing	Mutation score over broad automatic mutations; Stryker can fail the build below a configured threshold	Threshold set by the user	No	No	Tests only, by percentage	[12][13]
canfail (PyPI, 2026-08-31)	Declared breaks must turn a check red; green baseline first; anchor exactly once; syntax-only failures and runs without evidence are not catches; restore verified	Not applicable	No	No	Yes, for each declared break; no requirement that every check has one	[17]
didrun (PyPI, 2026-08-31)	A check must show positive evidence in its output that it ran (a count, a pattern, a written file)	Not applicable	No	No	Ran-or-not, per run	[18]
restore-verified (PyPI, 2026-08-31)	A mutated file is restored and the restore checked byte for byte; an outer sentinel detects a tree left modified after SIGKILL; no locking	Not applicable	No	No	Covers the restore step of a mutation harness	[19]
IEC 62304 / ISO 13485	Documented life cycle, change control, traceability, audited	Process, not numeric	Through configuration management, as a process	Through design and risk files, as a process	Through verification records, as a process	[20][21][24]

Approach	What it enforces	Direction of limits	Ties numbers to a version	Evidence or claim inventory	Proves its own checks fail	Source
Emuqu (this paper)	Constants-hash/version lock; claim register with closed vocabulary, test-existence, version match, retracted claims and discovery; per-file direction ratchets	Ceilings fall and <code>min_</code> floors rise, checked against a baseline commit	Yes, for numeric constants in two files, and the version is stored on each score	Yes, machine-checked fields with a per-entry wording boundary, versioned with the code	Yes: a plant required for every gate outside the build exemptions, preflight, wiring check, 125 targeted test mutations, static leak detector and tree lock	This paper

Emuqu is a consumer wellness app. It is not developed under IEC 62304 or ISO 13485 and claims no conformity with them. They appear in this table only because traceability and controlled change are what those standards address by process, and the mechanisms here address a narrow part of the same concern by build-time checks.

5. What Is New

Each statement gives only the part not found in a prior-art search made on 2026-10-08 (PyPI, npm, NuGet and GitHub; vendor, lint-rule, requirements-tool and documentation-lint tool documentation; and the literature cited in the references). Commit dates are author dates in US Central time (CST = UTC−6, CDT = UTC−5). Each form of a statement was publicly disclosed when it reached the public repository: on 2026-09-08 (first commit 9d89933) for forms committed before that date, and on its commit date for later forms, unless an earlier public disclosure is noted. Statement numbering follows v1.1. The section after each label is where the method is specified.

N1 (Section 3.1.1). A build gate that hashes only the comment-stripped numeric `static let` declarations of designated scoring-constant files and fails the build when that digest moves while the scoring version stays the same. The digest is SHA-256 over a per-file sorted list of `path:symbol=value` lines, truncated to 16 hexadecimal characters; the version compared is the same string stamped on every stored score (N2), checked against the version recorded beside the digest.

Builds on: hashing configuration or lockfiles to detect change, which is general practice; CI checks that fail when hashed content changes without a semantic-version bump were later proposed in maiconsouza89/agents-skills issue 130 (2026-09-17) [22] and pandas-studio/agent-team-plugins pull request 165 (2026-09-27) [23], both after Emuqu's public commit; adds selection of numeric declarations only, comment stripping, and a tie to the version stamped on stored user data.

Earliest dated record: flow-recovery dba6fcc, 2026-05-01 (first form, a precursor: a unit test that pinned the version string of a versioned scoring-parameters value and its six score-band values, failing if a value changed without a version change); emuqu-dev f2e6ed1, 2026-09-03 (current form, the hash gate).

N2 (Section 3.1.3). Persisting the scoring-version string on every stored score breakdown and showing it to the user. Records written before the field existed decode to an explicit `unversioned` sentinel rather than to the current version; the stamp is carried through every derived copy; and the score-detail screen renders the stored stamp, or "Scored before score versions were recorded", next to a fixed statement that the weights are calibrated, not outcome-validated.

Builds on: version identifiers on machine-learning models in model cards and registries [10][11]; adds a version stamped on each stored consumer score and shown to the user. No wearable vendor documentation found describes score versioning (searched: WHOOP, Oura, Garmin and Polar public documentation).

Earliest dated record: flow-recovery dba6fcc, 2026-05-01 (precursor: versioned scoring parameters, not persisted with scores); emuqu-dev f2e6ed1, 2026-09-03 (current form).

N3 (Section 3.1.2). A containment gate that confines a hand-set ranking-weight identifier to a single source directory. A number with no outcome evidence therefore cannot be displayed as a confidence anywhere else in the app without a visible rename.

Builds on: general lint rules restricting identifiers; adds confinement tied to an evidence-register entry. No prior instance found.

Earliest dated record: emuqu-dev f2e6ed1, 2026-09-03 (first and current form).

N4 (Section 3.2). A register of the score-adjusting heuristics in an app in which each entry must carry a non-empty `allowed_language` field and the register's scoring version must equal the code's. The `allowed_language` field states the user-facing wording the entry permits and forbids; because the versions must agree, every scoring-version change fails the build until the register is edited.

Builds on: sphinx-needs, which restricts item statuses to a configured list of names with descriptions and enforces required fields by schema [14]; Doorstop [15] and reqcov [16] (PyPI, 2026-09-04), which check that requirements are linked to tests and code. A closed status vocabulary, required fields and existence checks on cited tests are therefore not claimed. Adds a tie from the inventory to a code version, and a per-item wording boundary.

Earliest dated record: emuqu-dev f2e6ed1, 2026-09-03 (first and current form).

N5 (Section 3.2.2). A retracted-claims list kept inside the register whose strings fail the build if they reappear anywhere else. The scan covers every register entry and every Swift or Markdown file under the app sources and documentation, making the register the only place a corrected error may be written.

Builds on: requirements tools that keep an inventory in the repository (sphinx-needs, Doorstop, reqcov); adds a list of corrected phrasings enforced across the code and documentation. No prior instance found.

Earliest dated record: emuqu 9d89933, 2026-09-08 (first and current form; absent from emuqu-dev).

N6 (Section 3.2.2). Name-shape discovery of score-adjusting functions with a symmetric stale-claim check and a rising entry floor. Functions in the analysis layer whose names end in Adjustment, Penalty, Bonus, Modifier, Damper or Fraction must each be claimed by a register entry's `symbols` list or waived by name with a reason; every claimed or waived name must still exist; and the register's entry count is a floor that only rises.

Builds on: traceability tools that check links from requirements to code [14][16][15]; adds discovery of unregistered code by name shape, with a symmetric stale-claim check and a rising entry floor. No prior instance found.

Earliest dated record: emuqu-dev f2e6ed1, 2026-09-03 (first form: discovery and stale-claim check); emuqu 9d89933, 2026-09-08 (current form: entry-count floor).

N7 (Section 3.4.1). A planted-violation harness that refuses to report success while any `check_`/`enforce_` script in the scripts directory lacks a plant. It searches its own source for each gate's filename and fails the run if one is absent, so adding a gate without a plant fails the build. A named list exempts the two gates that need a build or simulator; the preflight gate is excluded from the name search as the helper library's self-test but carries its own plants.

Builds on: canfail (PyPI, 2026-08-31) [17], which breaks a file on purpose, requires the check to go red, confirms the check is green first, requires the anchor to match exactly once and verifies the restore; those rules are therefore not claimed. Adds completeness: a requirement that every check in the repository has a plant.

Earliest dated record: emuqu-dev 26fe7b1, 2026-08-28 (first form: plants, green first); emuqu-dev ba968cd, 2026-09-02 (current form: completeness check).

N8 (Section 3.4.2). A preflight gate that statically requires every gate's temp-file creation to go through a helper that exits non-zero on failure. Every call site must carry an `|| exit` guard on the same line, and every sourced helper path must resolve, so that gates fail closed instead of reporting clean after measuring nothing.

Builds on: didrun (PyPI, 2026-08-31) [18], which requires positive evidence in a check's output that it ran; adds a static check of the gate's own source for the temp-file failure class.

Earliest dated record: emuqu-dev 26fe7b1, 2026-08-28 (first and current form).

N9 (Section 3.4.3). A wiring meta-gate that fails when any `check_`/`enforce_` script's filename does not appear in the CI workflow file. A named exemption covers a gate invoked by another step. Together with N7 and N8 it makes "is wired into CI" a third separately enforced property of every gate, next to "goes red" and "fails closed"; the preflight and wiring meta-gates are themselves covered by planted violations.

Builds on: canfail [17] (goes red), didrun [18] (ran) and Semgrep's `ruleid:` test annotations [25], which prove that a rule fires on an example; adds a check that every gate is reached by CI. No prior instance found.

Earliest dated record: emuqu-dev ba968cd, 2026-09-02 (first and current form).

N10 (Section 3.5.1). A curated production-mutation harness in which each mutation names the single test suite that must catch it, a misclassified baseline is retried once, and any skip fails the run. The named suite is run alone; a baseline run classified as a build or runner failure (a build-failure banner, or no "Test Suite"/"Test Case" line in the log) is retried exactly once; and any skipped mutation (baseline not green, or mutation not applicable) fails the whole run.

Builds on: mutation testing in general [12][13][26][27], which is not claimed, and canfail [17], which also requires a green baseline and an anchor that matches exactly once, and scores a syntax error, or a run with no evidence it

ran, as not a catch; those rules are not claimed. Adds per-mutation suite attribution, a single classified retry, and skips that fail the run (in canfail such cases are reported without failing it).

Earliest dated record: emuqu-dev d0267b3 "Add mutation testing", 2026-08-28 (first form: anchor exactly once, green first); emuqu-dev 409a762, 2026-09-01 (current form: build/test classification, single retry, skips fail).

N11 (Section 3.5.2). A static leak detector that recovers each mutation's target and original text from the mutation harness's own source, combined with an atomic `mkdir` tree lock shared by both mutation harnesses. The detector handles `chr()`-escaped forms and fails if the original text is absent from its target file or if any case cannot be parsed. The lock lives in the git directory, records the holder's PID, refuses a second mutator while that PID is alive, takes over a dead holder's lock with a warning, and comes with a reader check, used by the local CI simulation, that refuses to run while a live mutator holds it.

Builds on: restore-verified (PyPI, 2026-08-31) [19], which verifies a restore byte for byte against a digest recorded beforehand, with an outer sentinel that detects a tree left modified after SIGKILL; adds detection that needs no manifest recorded before the run, and locking between mutators and readers.

Earliest dated record: emuqu-dev ba968cd, 2026-09-02 (first and current form).

N12 (Section 3.6.2). A budget ratchet over a directory of single-integer files in which the allowed direction is read from the filename. A `min_` prefix marks a floor that may only rise and any other name a ceiling that may only fall; a loosening passes only when a commit in the compared range carries a `budget-raise-ok: <path>` trailer naming that file; and a CI-set flag turns a missing baseline into a failure.

Builds on: jest-ratchet (npm, 2018) [8] and CoverageRatchet (NuGet, versions listed from 2026-05-05) [9], which ratchet coverage thresholds upward, and betterer [7] and SwiftLint baselines [6], which freeze existing results. Ratcheting as an idea is not claimed. Adds per-file direction from the filename, an audited per-file trailer, and a required baseline in CI.

Earliest dated record: emuqu-dev 17f1114, 2026-08-22 (first form: `min_` floors and trailer); emuqu-dev 6a7ff33, 2026-08-31 (current form: require-baseline flag).

N13 (Section 3.6.3). A documentation gate that rejects any Markdown line naming a ratcheting budget file next to a bare number. Prose must therefore point at the value instead of restating it.

Builds on: documentation link checkers in general; adds a rule tying prose to ratcheting values. No prior instance found.

Earliest dated record: emuqu-dev 8c3999d, 2026-08-26 (first and current form).

N14 (Section 3.8). Composing N1, N4, N6 and a lexical regulated-claim perimeter into one chain. A change to a constant in a designated scoring file forces a version change, the version change forces an edit of the evidence register that states each heuristic's permitted wording, and that wording is policed for prohibited vocabulary by the perimeter.

Builds on: the individual tool families of Section 4; adds the composition. No prior instance found.

Earliest dated record: emuqu-dev f2e6ed1, 2026-09-03 (first and current form).

canfail, didrun and restore-verified were published on PyPI on 2026-08-31 [17][18][19]. That is three days after the private first forms of N7, N8 and N10 (2026-08-28) and eight days before Emuqu's public commit (2026-09-08). The private dates show the methods were developed independently. They do not make the author's work the earlier disclosure, because the packages were public first. This section therefore claims only what those packages do not contain.

Prior work used, not presented as new.

- Version identifiers on software releases, as in semantic versioning [28].
- Git commit trailers as a convention.
- Traceability between requirements, risk controls and verification as required by medical-device standards [20][21][24].
- The HRV methods named in the register, which are cited to their published sources [1][2].
- The general observation that machine-learning and data systems accumulate configuration and documentation debt [29][30].

6. Verification

Each gate's ability to fail is itself checked (Section 3.4). The counts below are printed by the gates at e028039.

Table 6.1. Counts printed by the gates at e028039.

Check	Result at e028039
Constants hashed by the scoring-governance gate	155 (91 + 64)
Score-adjusting functions found by discovery	13: 12 claimed, 1 waived
Planted violations in the gate harness	64, with the every-gate-has-a-plant check applied to all gates except the two-entry build exemption and the preflight gate (which has its own plants)
Scripts scanned by the preflight gate	53, with 10 guarded temp-file uses
Gates reached by the full-CI workflow	44 named in the workflow, 1 invoked by the coverage step
Targeted test mutations parsed by the leak detector	125 of 125 parsed; no leaked mutation
Integer budgets changed across the 28 public commits	Every change in its tightening direction; no <code>budget-raise-ok</code> : trailer used

The analysis methods that the register classifies were also checked by a separate reimplementaion in Python, built from the Swift source at e028039 and run against the app's unit-test expectations and PhysioNet recordings; the results are in the cross-implementaion report (`Tools/validation/RESULTS.md`) [31].

7. Design Tradeoffs

Recorded baseline, not history. The hash lock compares the code with one recorded line, which keeps the gate stateless and readable; the recorded line itself is trusted, so edits to it are left to review.

Two files, literal declarations. The lock covers the numeric `static let` literals of two designated files. The composite weights live in a separate weights namespace in the general constants file and are governed through their register entry (`composite-weights-60-25-15`) rather than the hash; inline literals and expression-valued constants are outside it.

Truncated digest. Sixteen hexadecimal characters detect accidental change; they are not meant to resist deliberate collision.

Lexical checks. Ranking-weight confinement, name-shape discovery and the plant and wiring checks match names and filenames, which keeps them fast and deterministic at the cost of seeing only the shapes they look for. The register states this itself in its scope statement: "governance over a known failure class, not proof that nothing was omitted."

Structure, not truth. The register gate checks fields, links, versions and phrasings; whether a status is honest, and whether copy matches an entry's `allowed_language`, remain maintainer judgments that the version gate forces to be revisited.

Targeted, manual mutation. Curated mutations name the behavior at stake but cover only the behaviors listed, and the harness runs by hand because each case needs a simulator build; the static leak detector is its standing check on every push to main and every pull request.

Measured ceilings. Setting each ceiling at the measured value tolerates existing debt, which ADR 004 calls "debt inventories rather than assurance targets", in exchange for gates that can be satisfied on the day they land.

Scope. None of this verifies that the science is right; it establishes that numbers, versions, statuses, citations and wording stay consistent with one another and cannot change silently.

8. Provenance and Dates

8.1 Project origin

The project began as a general fitness app. Repository AI-Fitness-Coach (2025-02-16 to 2025-02-17, 19 commits, private) displayed Apple HealthKit heart rate and HRV (SDNN), streamed Polar H10 heart rate over Bluetooth and estimated VO₂max; it contains no RR-interval analysis. On 2025-02-22 (15:44 UTC) the author proposed, in a ChatGPT conversation, an app that would take an HRV reading from a chest strap worn overnight just before the user wakes (excerpt recovered through conversation-history retrieval on 2026-10-08). That thread is separate from the 2025 records of the author's design-audit method, which concern another project (Paper 5). The Emuqu design itself begins with the full-night capture problem statement of December 2025 (Paper 5, Section 4).

8.2 Repository lineage

Table 8.1. Repository lineage. All repositories are owned by the author and retained with full history.

Repository	Product name	First commit	Extent	Visibility
chrissharp80/AI-Fitness-Coach	(fitness app)	2025-02-16	19 commits to 2025-02-17	Private
chrissharp80/flow-recovery-old	Flow HRV	e5948bd, 2026-01-12	250 commits to 2026-01-26	Private
chrissharp80/flow-recovery-dev	Flow Recovery	c48eeb9, 2026-03-01	Squashed import; to 2026-03-15	Private
chrissharp80/flow-recovery	Flow Recovery	39bfcd4, 2026-03-15	196 commits to 2026-08-12	Private
chrissharp80/emuqu-dev	Emuqu	d4e5cf8, 2026-08-16	Squashed import; to 2026-09-04	Private
chrissharp80/emuqu	Emuqu	9d89933, 2026-09-08	Squashed import; current	Public (PolyForm Strict 1.0.0)

History has gaps from 2026-01-26 to 2026-03-01, 2026-08-12 to 2026-08-16 and 2026-09-04 to 2026-09-08, where each new repository began with a squashed import; a date at a squashed import is an upper bound for anything it contains. The pre-code design (v1 to v9.2 FINAL, completed 2026-01-08 21:06 CST) is described in Paper 5, Section 4: versions v1 and v3.0 have not been recovered; v2–v2.2 and v4.0–v9.1 survive in the author's Claude conversation record; v9.2 FINAL survives as a Google Drive document.

8.3 Dated records outside git

- Project documents uploaded to the author's Google Drive on 2026-05-10 (README, audit and refactor notes dated 2026-05-06) describe "Frozen Historical Scores", the copy linter's prohibited-terms list, and CI budget files for tech debt, SwiftLint and large files. They show that budget files and a regulated-claim perimeter existed by May 2026. They do not describe the direction-from-filename rule, the trailer or any other statement in Section 5.

No other record outside git dates a method in this paper.

8.4 Public disclosures

- 2026-01-27: TestFlight beta opened (over 1,500 sessions by 2026-03-06; 45 testers and over 6,500 sessions by August 2026). A public TestFlight join link appeared in the project README by May 2026.
- 2026-05-19: Substack article "A Powerfully Lazy Man's Way to Better Health" (Paper 1 lists what it disclosed).
- 2026-08-10: Substack article "How I Build Production Software by Directing AI" (Paper 5 lists what it disclosed).
- 2026-09-08: public repository github.com/chrissharp80/emuqu (first commit 9d89933).

- 2026-10-08: these white papers.

Neither Substack article disclosed any method in Section 5 of this paper.

8.5 Authorship

Chris Sharp is the sole author of these papers and of the design, and the only person who commits to the repositories above. The code was written by AI coding assistants under his direction, and some commits carry "Claude" as author or co-author; he supplied the requirements, design direction, review, testing and acceptance, and wrote no code by hand. He developed the methods described here independently and did not learn them from anyone.

8.6 Document history

v1.0, v1.1, v1.2 and v1.3 all published 2026-10-08. v1.1 added prior-art comparison and provenance; v1.2 restructured method-first and moved verification detail to the cross-implementation report; v1.3 applied an editorial review (series-wide format, corrected internal references, tightened claims).

References

Items marked [S] were seen only through a search-engine extract on 2026-10-08.

1. Task Force of the European Society of Cardiology and the North American Society of Pacing and Electrophysiology. Heart rate variability: standards of measurement, physiological interpretation and clinical use. *Circulation*. 1996;93(5):1043–1065. doi:10.1161/01.CIR.93.5.1043
2. Plews DJ, Laursen PB, Stanley J, Kilding AE, Buchheit M. Training adaptation and heart rate variability in elite endurance athletes. *Sports Med*. 2013;43(9):773–781. doi:10.1007/s40279-013-0071-8
3. Peake JM, Kerr G, Sullivan JP. A critical review of consumer wearables, mobile applications, and equipment for providing biofeedback, monitoring stress, and sleep in physically active populations. *Front Physiol*. 2018;9:743. doi:10.3389/fphys.2018.00743
4. Billman GE. The LF/HF ratio does not accurately measure cardiac sympatho-vagal balance. *Front Physiol*. 2013;4:26. doi:10.3389/fphys.2013.00026
5. Impellizzeri FM, Tenan MS, Kempton T, Novak A, Coutts AJ. Acute:chronic workload ratio: conceptual issues and fundamental pitfalls. *Int J Sports Physiol Perform*. 2020;15(6):907–913. doi:10.1123/ijsp.2019-0864
6. Realm. SwiftLint: baseline option (`--baseline`, `--write-baseline`) [Internet]. <https://github.com/realm/SwiftLint> (accessed 2026-10-08). [S]
7. Spence C (phenomnomnominal). betterer: incremental improvement test runner [Internet]. <https://github.com/phenomnomnominal/betterer> (accessed 2026-10-08).
8. markis (GitHub user). jest-ratchet: "Ratchet up code coverage — keep test coverage going only one direction — up" [Internet]. npm package, first published 2018-01-15. 2018. <https://www.npmjs.com/package/jest-ratchet>; source <https://github.com/markis/jest-ratchet> (accessed 2026-10-08).

9. michaelglass (NuGet owner). CoverageRatchet: per-file coverage enforcement with automatic threshold ratcheting from Cobertura XML [Internet]. NuGet package; versions listed from 2026-05-05. 2026. <https://www.nuget.org/packages/CoverageRatchet> (accessed 2026-10-08).
10. Mitchell M, Wu S, Zaldivar A, Barnes P, Vasserman L, Hutchinson B, et al. Model cards for model reporting. In: Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '19). New York: ACM; 2019. p. 220–229. doi:10.1145/3287560.3287596
11. Zaharia M, Chen A, Davidson A, et al. Accelerating the machine learning lifecycle with MLflow. IEEE Data Eng Bull. 2018;41(4):39–45.
12. Coles H, Laurent T, Henard C, Papadakis M, Ventresque A. PIT: a practical mutation testing tool for Java (demo). In: Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA 2016). New York: ACM; 2016. p. 449–452. doi:10.1145/2931037.2948707
13. Stryker Mutator. Stryker documentation: mutation score and thresholds [Internet]. <https://stryker-mutator.io> (accessed 2026-10-08).
14. useblocks. sphinx-needs: Sphinx extension for requirements and specification items (configuration option `needs_statuses`; field schema validation) [Internet]. <https://github.com/useblocks/sphinx-needs> (accessed 2026-10-08).
15. Doorstop contributors. Doorstop: requirements management using version control [Internet]. <https://github.com/doorstop-dev/doorstop> (accessed 2026-10-08). [S]
16. reqcov contributors. reqcov: requirements coverage for pull requests [Internet]. Python package, version 0.1.0 released on PyPI 2026-09-04. 2026. <https://pypi.org/project/reqcov>; source <https://github.com/Antoine005/reqcov> (accessed 2026-10-08).
17. Megapixel99 (GitHub user). canfail: "Break the thing on purpose and check that your check notices." [Internet]. Python package, version 0.1.0 released on PyPI 2026-08-31. 2026. <https://pypi.org/project/canfail>; source <https://github.com/Megapixel99/canfail> (accessed 2026-10-08).
18. Megapixel99 (GitHub user). didrun: "An exit code cannot tell you whether anything happened." [Internet]. Python package, version 0.1.0 released on PyPI 2026-08-31 (also on npm as @megapixel99/didrun). 2026. <https://pypi.org/project/didrun>; source <https://github.com/Megapixel99/didrun> (accessed 2026-10-08).
19. Megapixel99 (GitHub user). restore-verified: "Temporarily modify a file, survive the signal, and prove the tree came back." [Internet]. Python package, version 0.0.1 released on PyPI 2026-08-31 (also on npm). 2026. <https://pypi.org/project/restore-verified>; source <https://github.com/Megapixel99/restore-verified> (accessed 2026-10-08).
20. International Electrotechnical Commission. IEC 62304:2006+AMD1:2015. Medical device software — Software life cycle processes. Geneva: IEC; 2015.
21. International Organization for Standardization. ISO 13485:2016. Medical devices — Quality management systems — Requirements for regulatory purposes. Geneva: ISO; 2016.
22. maiconsouza89 (GitHub user). agents-skills issue 130: CI check failing when hashed content changes without a semantic-version bump [Internet]. GitHub; opened 2026-09-17. 2026. <https://github.com/maiconsouza89/agents-skills/issues/130> (accessed 2026-10-08).
23. pandas-studio (GitHub organization). agent-team-plugins pull request 165: CI check failing when hashed content changes without a semantic-version bump [Internet]. GitHub; opened 2026-09-27. 2026. <https://github.com/pandas-studio/agent-team-plugins/pull/165> (accessed 2026-10-08).
24. International Organization for Standardization. ISO 14971:2019. Medical devices — Application of risk management to medical devices. Geneva: ISO; 2019.

25. Semgrep. Testing rules (`ruleid:` and `ok:` annotations) [Internet]. <https://semgrep.dev/docs/writing-rules/testing-rules> (accessed 2026-10-08). [S]
26. Jia Y, Harman M. An analysis and survey of the development of mutation testing. *IEEE Trans Softw Eng.* 2011;37(5):649–678. doi:10.1109/TSE.2010.62
27. Petrović G, Ivanković M. State of mutation testing at Google. In: *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP 2018)*. New York: ACM; 2018. p. 163–171. doi:10.1145/3183519.3183521
28. Preston-Werner T. Semantic Versioning 2.0.0 [Internet]. <https://semver.org> (accessed 2026-10-08).
29. Sculley D, Holt G, Golovin D, et al. Hidden technical debt in machine learning systems. In: *Advances in Neural Information Processing Systems 28 (NeurIPS 2015)*. 2015.
30. Breck E, Cai S, Nielsen E, Salib M, Sculley D. The ML test score: a rubric for ML production readiness and technical debt reduction. In: *2017 IEEE International Conference on Big Data*. IEEE; 2017. p. 1123–1132.
31. Sharp C. Cross-implementation report: Emuqu beat-interval methods re-implemented in Python and run against PhysioNet recordings. Emuqu repository, Tools/validation/RESULTS.md; run 2026-10-08. PhysioNet data: slpdb 1.0.0, capslpdb 1.0.0, mitdb 1.0.0, nsr2db 1.0.0.

Appendix A: Parameters

Table A.1. Parameters and counts at e028039.

Name	Value	Unit	Role
Hashed source files	2	files	Recovery-score constants file and HRV thresholds file
Hashed constants	155 (91 + 64)	declarations	Count at this revision
Constant regex	<code>static let\s+(\w+)\s*:?\\s*[\w<>\\[\],.]*=\\s*([-\d.]+)</code>	—	Extracts symbol and literal value
Comment strip (hash)	<code>//[^\n]*</code> → empty	—	Line comments only
Hash line format	<code><path>:<symbol>=<value>\n</code>	UTF-8	One per constant, per-file sorted
Digest	SHA-256, first 16 hexadecimal characters	64 bits	Constants fingerprint
Baseline line	<code><version> <hash></code>	—	Recorded pair
Current version	v3.1.oct2026	—	Version this build computes
Legacy sentinel	<code>unversioned</code>	—	Decoded value for unstamped records
Ranking-key identifier	<code>rankingWeight</code>	—	Allowed only in the cause-detection directory
Register entries	19	entries	Count at this revision
Status vocabulary	validated, supported-transfer, awaiting-validation, scheduled-for-removal	—	Keys of <code>_status_meanings</code>

Name	Value	Unit	Role
Required non-empty fields	affects, evidence, allowed_language	—	Per entry
Retracted strings	3	strings	Scanned in entries, app sources and docs (.swift, .md)
Discovery suffixes	Adjustment, Penalty, Bonus, Modifier, Damper, Fraction	—	Function-name shape
Discovery exclusions	names ending Descriptions or Message; cause-detection directory	—	Not score inputs
Discovered functions	13 (12 claimed, 1 waived)	functions	At this revision
Gate scripts	45	scripts	<code>check_*</code> and <code>enforce_*</code>
Planted violations	64	plants	Planted-violation harness
Plant-completeness exclusions	coverage-floor gate, Thread Sanitizer gate (need a build or simulator); preflight gate (helper library's self-test, has its own plants)	—	Not searched for by name
Targeted test mutations	125	mutations	Test-mutation harness
Build-failure retry	1	retry	Baseline suite run only
Wiring exemptions	1	gate	Coverage gate runs inside the coverage step
Preflight-scanned scripts	53	scripts	All shell scripts except the preflight gate
Gate exit codes	0 clean, 1 violation, 2 configuration error	—	Scoring, register, wiring gates
Helper exit code	70 (EX_OSERR)	—	Temp-file or empty-input failure
Lock-held exit code	75 (EX_TEMPFAIL)	—	Another mutator is live
Trailer	<code>budget-raise-ok: <path> <reason></code>	—	Acknowledged loosening; case-insensitive key
Floor prefix	<code>min_</code>	—	Budget may only rise
Require-baseline flag	<code>BUDGET_REQUIRE_BASELINE</code>	env var	Missing baseline becomes a failure
Doc-quote number regex	<code>(?![\w.])(\d{2,6})(?![\w.%])</code>	—	Bare number near a budget filename
Doc-quote window	current line + next line	lines	Where the number is sought
macOS billing multiplier	10	× Linux	CI cost basis

Name	Value	Unit	Role
Monthly plan	3,000 billable (about 300 macOS)	minutes	CI cost basis
Full macOS run	~108 wall / ~1,080 billable	minutes	Measured by the repository

Appendix B: Pseudocode

B.1 Constants-hash version lock

```

function scoring_governance():
    problems = []
    for file in all_swift_files(APP_SOURCES):
        if file not under CAUSE_DETECTION_DIR:
            for (n, line) in lines(file):
                if "rankingWeight" in line: problems.add(file, n)
    h = sha256()
    count = 0
    for path in [RECOVERY_CONSTANTS, HRV_THRESHOLDS]:           # fixed order
        if not exists(path): exit 2
        src = regex_replace(read(path), "//[^\n]*", "")
        pairs = sorted(regex_findall(src, CONST_RE))             # (symbol, value)
        for (sym, val) in pairs:
            h.update(utf8(path + ":" + sym + "=" + val + "\n"))
            count += 1
    current_hash = hex(h)[0:16]
    version = regex_group(read(VERSION_FILE), 'static let current\s*=\s*"([^\n]+)"') or
exit 2
    if not exists(BASELINE): exit 2
    (rec_version, rec_hash) = split_whitespace(read(BASELINE))
    if current_hash != rec_hash and version == rec_version:
        problems.add("constants changed without a version change")
    print(count, current_hash, version)
    exit 1 if problems else 0

decode ScoreBreakdown(record):
    scoringVersion = record.get("scoringVersion") ?? "unversioned"

```

B.2 Science register gate

```
function science_register():
    reg = parse_json(REGISTER) or exit 2
    VALID = keys(reg._status_meanings); if empty: exit 2
    problems = []; seen = {}
    for e in reg.entries:
        if e.id in seen: problems.add(dup)
        seen.add(e.id)
        if e.status not in VALID: problems.add(status)
        for f in [affects, evidence, allowed_language]: if empty(e[f]): problems.add(f)
        for t in e.tests or []: if not exists(t): problems.add(t)
    if reg.scoring_version != read_current_version(): problems.add(version)
    for c in reg._retracted_claims.claims:
        for e in reg.entries, f in [evidence, affects, allowed_language, notes]:
            if c in join(e[f]): problems.add(c)
        for file in files(APP_SOURCES, DOCS) with ext in {swift, .md}:
            if c in read(file): problems.add(file, c)
    discovered = {}
    for file in swift_files(ANALYSIS_DIR) not under CAUSE_DETECTION_DIR:
        for name in findall("func\s+([a-zA-Z]+
(?:Adjustment|Penalty|Bonus|Modifier|Damper|Fraction))\b"):
            if not name.endswith("Descriptions" or "Message"): discovered.add(name)
    claimed = union(e.symbols for e in entries)
    waived = {k in reg._waived_symbols if not k.startswith("_")}
    for n in discovered - claimed - waived: problems.add(unclassified n)
    for n in (claimed u waived) - discovered: problems.add(stale n)
    print tally
    if problems: exit 1
    if len(entries) < read_int(REGISTER_FLOOR): exit 1
    exit 0
```

B.3 Planted-violation harness

```
for (name, gate, file, mutation) in PLANTS:
    if run(gate) != 0: SKIP; continue
    backup(file)
    if not run_python(mutation): restore(); SKIP; continue
    code = run(gate); restore_all()
    if code != 0: PASS else: FAIL
unplanted = [g for g in glob(check_*.sh, enforce_*.sh)
              if g != "check_gate_preflight.sh" and g not in NEEDS_BUILD
              and basename(g) not in read(this_script)]
if unplanted: exit 1
exit 1 if FAIL > 0 else 0
```

B.4 Preflight and wiring

```
preflight:
  for s in shell_scripts except self:
    for line in non_comment_lines(s):
      if matches("(?!gate_)\bmktemp\b") and "gate_mktemp" not in line: fail
      if "gate_mktemp" in line and "|| exit" not in line: fail
      if line sources "$(dirname "$0")/X" and not exists(scripts/X): fail

wiring:
  wf = read(WORKFLOW) or exit 2
  for g in gates (check_*/enforce_*) not in NOT_A_STEP:
    if basename(g) not substring of wf: unwired.add(g)
  exit 1 if unwired else 0
```

B.5 Targeted test mutations and leak detector

```
run_suite(s): xcodebuild test -only-testing s
  -> 0 passed | 2 if build banner or no "Test Suite|Test Case" | 1 otherwise
for (name, suite, file, mutation) in MUTATIONS:
  b = run_suite(suite); if b == 2: b = run_suite(suite)
  if b != 0: SKIP; continue
  backup(file); if not apply(mutation): restore(); SKIP; continue
  r = run_suite(suite); restore()
  if r != 0: PASS else: FAIL
exit 1 if FAIL or SKIP else 0

leak_detector:
  for case in split(read(MUTATION_SCRIPT), '\nmutate ')[1:]:
    path = first double-quoted app-directory .swift path in case
    orig = group(o='(.*)';assert) or group(s.count('(.*)')\s*[=><])
    if not path or not orig: unparsed.add; continue
    orig = unescape(orig); orig = resolve("'" + chr(N) + "'" -> char N)
    if "chr(" in orig or "'" + " in orig: unparsed.add; continue
    if not exists(path) or orig not in read(path): leaked.add
  exit 1 if leaked or unparsed else 0

tree_lock_acquire(owner):
  if mkdir(LOCK) fails:
    if pid_alive(read(LOCK/pid)): exit 75
    warn "stale lock: previous run never restored"; rm LOCK; mkdir(LOCK) or exit 70
  write LOCK/owner, LOCK/pid

tree_lock_require_unmutated(who):          # reader side
  if exists(LOCK) and pid_alive(read(LOCK/pid)): exit 75
  if exists(LOCK): warn "stale lock"; rm LOCK
```

B.6 Direction-aware ratchet

```

base = arg or ("origin/" + GITHUB_BASE_REF) or BUDGET_BASE_REF
if not base or base == 40 zeros or not resolvable(base):
    if BUDGET_REQUIRE_BASELINE: exit 1 else: exit 0
acks = [path for msg in commit_messages(base..HEAD)
        for path in regex_i("^budget-raise-ok:\s*(\S+)", msg)]
for f in sorted(CI_DIR/*.txt):
    cur = int_or_skip(read(f)); prev = int_or_skip(git_show(base, f)) or skip
    if cur == prev: continue
    loosened = (cur < prev) if basename(f).startswith("min_") else (cur > prev)
    if loosened and f not in acks: violations.add(f, prev, cur)
exit 1 if violations else 0

count_for_budget(src):
    src = blank_block_comments_keep_newlines(src)
    src = blank_line_comments_same_length(src)
    src = blank(PLATFORM_SINGLETON.shared)
    return count(pattern, src)

```

Appendix C: Source Map

Table C.1. Where each method lives in the repository.

Method	Repository paths
Constants-hash lock, ranking- weight containment	scripts/check_scoring_governance.sh, .ci/scoring_constants_hash.txt (baseline), Emuqu/Sources/Utilities/Constants+RecoveryScore.swift, Emuqu/Sources/Analysis/HRVThresholds.swift
Version string and sentinel	Emuqu/Sources/Analysis/ScoringVersion.swift
Version persisted on breakdowns	Emuqu/Sources/Analysis/RecoveryScoreCalculator.swift, Emuqu/Sources/Analysis/VitalsScoring.swift, Emuqu/Sources/Storage/CloudKitSyncSupport.swift
Version shown	Emuqu/Sources/Views/RecoveryScoreDetailView+Sections.swift
Version given to the assistant	Emuqu/Sources/Assistant/Facts/AppFactResolver.swift, Emuqu/Sources/Assistant/Context/ContextBuilder.swift
Composite weights (governed by the register)	Emuqu/Sources/Utilities/Constants.swift
Science register and gate	Tools/science_register/register.json, scripts/check_science_register.sh, register floor file in .ci/
Regulated-claim perimeter (Paper 3)	Tools/copy_linter/lint.py, Tools/copy_linter/prohibited_terms.json, scripts/check_perimeter_sync.sh
Planted violations	scripts/verify_gates_fail.sh

Method	Repository paths
Fail-closed helpers and tree lock	<code>scripts/lib/preflight.sh</code> , <code>scripts/check_gate_preflight.sh</code> , <code>scripts/simulate_ci.sh</code> (lock reader)
Wiring	<code>scripts/check_gates_wired.sh</code> , <code>.github/workflows/ci.yml</code>
Targeted test mutations and leak backstop	<code>scripts/verify_tests_fail.sh</code> , <code>scripts/check_no_leaked_mutations.sh</code>
Ratchets	<code>docs/adr/004-ratcheted-budgets.md</code> , <code>scripts/check_budget_monotonicity.sh</code> , <code>scripts/enforce_tech_debt_budgets.sh</code> , <code>scripts/enforce_swifflint_budget.sh</code> , budget files in <code>.ci/</code>
Documentation constants rule	<code>scripts/check_doc_links.sh</code>
CI posture and evidence	<code>docs/CI_POSTURE.md</code> , <code>.github/workflows/gates.yml</code> , <code>.github/workflows/ci.yml</code> , <code>scripts/evidence_report.sh</code> , <code>Makefile</code>